

Miscellaneous Reflection Cleanup

Document #: P3795R0 [[Latest](#)] [[Status](#)]
Date: 2025-07-15
Project: Programming Language C++
Audience: EWG
Reply-to: Barry Revzin
<barry.revzin@gmail.com>

Contents

[1 Introduction](#)

[2 Proposal](#)

[2.1 Missing Predicates](#)

[2.2 Scope Identification](#)

[2.3 data_member_spec API](#)

[2.4 Annotations on Function Parameters](#)

[2.5 Missing Metafunctions](#)

[2.6 Inconsistent Error-Handling API](#)

[2.7 Specifying Error-Handling More Precisely](#)

[3 Wording](#)

[4 References](#)

§ 1 Introduction

At the Sofia meeting, [\[P2996R13\]](#) ([Reflection for C++26](#)), [\[P3394R4\]](#) ([Annotations for Reflection](#)), [\[P3293R3\]](#) ([Splicing a base class subobject](#)), [\[P3491R3\]](#) ([define static {string,object,array}](#)), [\[P3096R12\]](#) ([Function Parameter Reflection in Reflection for C++26](#)), and [\[P3560R2\]](#) ([Error Handling in Reflection](#)) were all adopted. Because these were all (somewhat) independent papers that were adopted at the same time, there were a few inconsistencies that were introduced. Some gaps in coverage. Some inconsistent APIs. This papers just seeks to correct a bunch of those little issues.

§ 2 Proposal

This isn't really one proposal, per se, as much as it is a number of very small proposals.

§ 2.1 Missing Predicates

[\[P2996R13\]](#) introduces a lot of unary predicates to help identify what a reflection represents. But there are a few simple ones that are missing:

- `is_inline(r)` — for identifying inline namespaces, and variables and functions declares inline (implicitly or explicitly)
- `is_constexpr(r)` - for identifying functions and variables declared `constexpr`
- `is_consteval(r)` - for identifying functions (and maybe eventually variables) declared `consteval`

These are all pretty simple predicates.

Notably, I'm suggesting `is_consteval` and not `is_immediate_function`. That's because you always know, even from inside of a function, whether or not it is *declared* `consteval`. But some `constexpr` functions can escalate — so what answer do you give from inside of a function that you're asking about? Moreover, we don't even have a good way of erroring in that context, since by making such a check either non-constant (or throw), we might cause the function to escalate, which may change its behavior. It's a complicated question, that needs to be carefully considered, whereas these three predicates are simple.

§ 2.2 Scope Identification

[P2996R13]'s mechanism for access checking relies on `std::meta::access_context`. However, that facility exposes another interesting bit of functionality — albeit in a very indirect way. You can identify what class you're currently in:

```
consteval auto current_class(std::meta::info scope = std::meta::access_
    context::current().scope())
-> std::meta::info
{
    while (true) {
        if (is_type(scope)) {
            return scope;
        }

        if (is_namespace(scope)) {
            throw std::meta::exception(/* ... */);
        }

        scope = parent_of(scope);
    }
}
```

Given that this is a useful (and occasionally asked for) piece of information, we should just provide it directly, rather than having this API proliferate.

The same is true for `current_function` and `current_namespace`. The only open question in my mind is whether there are situations in which you'd want something like a `nearest_enclosing_class_or_namespace`.

§ 2.3 data_member_spec API

The current API for `std::meta::define_aggregate` requires calls to `std::meta::data_member_spec`, which currently looks like this:

```
constexpr info data_member_spec(info type, data_member_options options);
```

When we originally proposed this API, we allowed the *name* of a non-static data member to be omitted. Doing so was akin to asking the implementation to create a unique name for you. However, that changed in [\[P2996R8\]](#) such that if `options.name` were not provided then the data member had to be an unnamed bit-field (which meant that `options.width` had to be provided). That means that while we originally envisioned it being possible to implement tuple like this:

```
constexpr {
    define_aggregate(
        ^^storage,
        {data_member_spec(^^Ts)...}
    );
}
```

That's no longer actually possible. You always have to provide *some* member of options. So tuple looks more like this (note that you're allowed to have multiple members named `_` now):

```
constexpr {
    define_aggregate(
        ^^storage,
        {data_member_spec(^^Ts, {.name="_"})...}
    );
}
```

As such, it looks a little odd to have the type off by itself like that. It's true that you *always* need to provide a type, but why is it special? Let's face it, approximately nobody is going to be creating unnamed bit-fields, so the name is practically always present too.

Let's just make the API more uniform:

```
struct data_member_options {
    struct name-type { // exposition only
        // ...
    };

+   info type;
    optional<name-type> name;
    optional<int> alignment;
    optional<int> bit_width;
    bool no_unique_address = false;
};
```

```

- consteval info data_member_spec(info type, data_member_options
  options);
+ consteval info data_member_spec(data_member_options options);

```

Additionally, while adding *attributes* is a difficult question, adding *annotations* is actually much simpler. With the adoption of [\[P3394R4\]](#), we should also allow you to add annotations to your generated members.

So the full change is really:

```

struct data_member_options {
    struct name-type { // exposition only
        // ...
    };

+   info type;
    optional<name-type> name;
    optional<int> alignment;
    optional<int> bit_width;
    bool no_unique_address = false;
+   vector<info> annotations = {};
};

- consteval info data_member_spec(info type, data_member_options
  options);
+ consteval info data_member_spec(data_member_options options);

```

§ 2.4 Annotations on Function Parameters

[\[P3394R4\]](#) ([Annotations for Reflection](#)) and [\[P3096R12\]](#) ([Function Parameter Reflection in Reflection for C++26](#)) were independent proposals adopted at the same time. The former gave us annotations, but the latter gave us the ability to introspect function parameters. Neither could really, directly add support for adding annotations onto function parameters. So, as a result, we don't have them.

But there isn't any particular reason why we shouldn't support this:

```

auto f([[=1]] int x) -> void;

constexpr auto a = annotations_of(
    parameters_of(^^f)[0]
)[0];
static_assert([: constant_of(a) :] == 1);

```

§ 2.5 Missing Metafunctions

[\[P1317R2\]](#) ([Remove return type deduction in std::apply](#)) was also adopted in Sofia. It added three new type traits, but none of us were aware of that paper, much less expected it to be adopted, so we neglected to add consteval metafunction equivalents to those three type traits:

```
constexpr bool is_applicable_type(info fn, info tuple);
constexpr bool is_nothrow_applicable_type(info fn, info tuple);
constexpr info apply_result(info fn, info tuple);
```

§ 2.6 Inconsistent Error-Handling API

[P2996R13]’s approach to error-handling was to add a “Constant When” specification to every function. Failing to meet that condition resulted in failing to be a constant expression.

Every other paper in the reflection constellation followed that same path.

However, [P3560R2] changes the error-handling approach to instead throw an object of type `std::meta::exception`. Its wording changed most of the functions in [P2996R13] — but it both did not change the error-handling for the type traits and it also neglected to be clairvoyant enough to change the error-handling for all of the other functions added by all of the other reflection papers.

That is:

Paper	Functions
[P3394R4]	<code>annotations_of</code> and <code>annotations_of_with_type</code>
[P3293R3]	<code>subobjects_of</code>
[P3491R3]	<code>reflect_constant_array</code>
[P3096R12]	<code>parameters_of</code> , <code>variable_of</code> , and <code>return_type_of</code>

All of these functions should just `throw` as well. That’s a pretty straightforward wording change.

§ 2.7 Specifying Error-Handling More Precisely

Currently, [P3560R2] specifies nothing about the contents of the exceptions being thrown on failure from various reflection functions. That’s largely as it should be — we really don’t need to specify the message, for instance.

But one thing that `std::meta::exception` gives you is a `from()` accessor that tells you which operation failed. And that we *should* specify — providing some front-matter sentence that says that when a function or function template `F` in `<meta>` is specified to throw a `meta::exception`, that `from()` is `^^F`.

§ 3 Wording

Extend what an annotation can represent in `[dcl.attr.annotation]`:

- 1 An annotation may be applied to any declaration of a type, type alias, variable, function, `function parameter`, namespace, enumerator, *base-specifier*, or non-static data member.

The synopsis change for [meta.syn] is:

```
#include <initializer_list>

namespace std::meta {
    using info = decltype(^:);

    // [meta.reflection.operators], operator representations
    enum class operators {
        see below;
    };
    using enum operators;
    consteval operators operator_of(info r);
    consteval string_view symbol_of(operators op);
    consteval u8string_view u8symbol_of(operators op);

    // [meta.reflection.names], reflection names and locations
    consteval bool has_identifier(info r);

    consteval string_view identifier_of(info r);
    consteval u8string_view u8identifier_of(info r);

    consteval string_view display_string_of(info r);
    consteval u8string_view u8display_string_of(info r);

    consteval source_location source_location_of(info r);

    // [meta.reflection.queries], reflection queries
    consteval info type_of(info r);
    consteval info object_of(info r);
    consteval info constant_of(info r);

    consteval bool is_public(info r);
    consteval bool is_protected(info r);
    consteval bool is_private(info r);

    consteval bool is_virtual(info r);
    consteval bool is_pure_virtual(info r);
    consteval bool is_override(info r);
    consteval bool is_final(info r);

    consteval bool is_deleted(info r);
    consteval bool is_defaulted(info r);
    consteval bool is_user_provided(info r);
    consteval bool is_user_declared(info r);
    consteval bool is_explicit(info r);
    consteval bool is_noexcept(info r);
    + consteval bool is_inline(info r);
    + consteval bool is_constexpr(info r);
    + consteval bool is_consteval(info r);

    consteval bool is_bit_field(info r);
    consteval bool is_enumerator(info r);
    consteval bool is_annotation(info r);
```

```
constexpr bool is_const(info r);
constexpr bool is_volatile(info r);
constexpr bool is_mutable_member(info r);
constexpr bool is_lvalue_reference_qualified(info r);
constexpr bool is_rvalue_reference_qualified(info r);

constexpr bool has_static_storage_duration(info r);
constexpr bool has_thread_storage_duration(info r);
constexpr bool has_automatic_storage_duration(info r);

constexpr bool has_internal_linkage(info r);
constexpr bool has_module_linkage(info r);
constexpr bool has_external_linkage(info r);
constexpr bool has_c_language_linkage(info r);
constexpr bool has_linkage(info r);

constexpr bool is_complete_type(info r);
constexpr bool is_enumerable_type(info r);

constexpr bool is_variable(info r);
constexpr bool is_type(info r);
constexpr bool is_namespace(info r);
constexpr bool is_type_alias(info r);
constexpr bool is_namespace_alias(info r);

constexpr bool is_function(info r);
constexpr bool is_conversion_function(info r);
constexpr bool is_operator_function(info r);
constexpr bool is_literal_operator(info r);
constexpr bool is_special_member_function(info r);
constexpr bool is_constructor(info r);
constexpr bool is_default_constructor(info r);
constexpr bool is_copy_constructor(info r);
constexpr bool is_move_constructor(info r);
constexpr bool is_assignment(info r);
constexpr bool is_copy_assignment(info r);
constexpr bool is_move_assignment(info r);
constexpr bool is_destructor(info r);

constexpr bool is_function_parameter(info r);
constexpr bool is_explicit_object_parameter(info r);
constexpr bool has_default_argument(info r);
constexpr bool has_ellipsis_parameter(info r);

constexpr bool is_template(info r);
constexpr bool is_function_template(info r);
constexpr bool is_variable_template(info r);
constexpr bool is_class_template(info r);
constexpr bool is_alias_template(info r);
constexpr bool is_conversion_function_template(info r);
constexpr bool is_operator_function_template(info r);
constexpr bool is_literal_operator_template(info r);
constexpr bool is_constructor_template(info r);
constexpr bool is_concept(info r);
```

```

consteval bool is_value(info r);
consteval bool is_object(info r);

consteval bool is_structured_binding(info r);

consteval bool is_class_member(info r);
consteval bool is_namespace_member(info r);
consteval bool is_nonstatic_data_member(info r);
consteval bool is_static_member(info r);
consteval bool is_base(info r);

consteval bool has_default_member_initializer(info r);

consteval bool has_parent(info r);
consteval info parent_of(info r);

consteval info dealias(info r);

consteval bool has_template_arguments(info r);
consteval info template_of(info r);
consteval vector<info> template_arguments_of(info r);
consteval vector<info> parameters_of(info r);
consteval info variable_of(info r);
consteval info return_type_of(info r);

// [meta.reflection.access.context], access control context
struct access_context;

// [meta.reflection.access.queries], member accessability queries
consteval bool is_accessible(info r, access_context ctx);
consteval bool has_inaccessible_nonstatic_data_members(info r, access_context ctx);
consteval bool has_inaccessible_bases(info r, access_context ctx);
consteval bool has_inaccessible_subobjects(info r, access_context ctx);

+ // [meta.reflection.scope], scope identification
+ consteval info current_function();
+ consteval info current_class();
+ consteval info current_namespace();

// [meta.reflection.member.queries], reflection member queries
consteval vector<info> members_of(info r, access_context ctx);
consteval vector<info> bases_of(info type, access_context ctx);
consteval vector<info> static_data_members_of(info type, access_context ctx);
consteval vector<info> nonstatic_data_members_of(info type, access_context ctx);
consteval vector<info> subobjects_of(info type, access_context ctx);
consteval vector<info> enumerators_of(info type_enum);

// [meta.reflection.layout], reflection layout queries
struct member_offset;
consteval member_offset offset_of(info r);
consteval size_t size_of(info r);

```

```

constexpr size_t alignment_of(info r);
constexpr size_t bit_size_of(info r);

// [meta.reflection.extract], value extraction
template<class T>
    constexpr T extract(info);

// [meta.reflection.substitute], reflection substitution
template<class R>
    concept reflection_range = see below;

template<reflection_range R = initializer_list<info>>
    constexpr bool can_substitute(info templ, R&& arguments);
template<reflection_range R = initializer_list<info>>
    constexpr info substitute(info templ, R&& arguments);

// [meta.reflection.result], expression result reflection
template<class T>
    constexpr info reflect_constant(const T& value);
template<class T>
    constexpr info reflect_object(T& object);
template<class T>
    constexpr info reflect_function(T& fn);

// [meta.reflection.array], promoting to static storage arrays
template<ranges::input_range R>
    constexpr info reflect_constant_string(R&& r);

template<ranges::input_range R>
    constexpr info reflect_constant_array(R&& r);

// [meta.reflection.define.aggregate], class definition generation
struct data_member_options;
- constexpr info data_member_spec(info type, data_member_options
  options);
+ constexpr info data_member_spec(data_member_options options);
constexpr bool is_data_member_spec(info r);
template<reflection_range R = initializer_list<info>>
    constexpr info define_aggregate(info type_class, R&&);

// associated with [meta.unary.cat], primary type categories
constexpr bool is_void_type(info type);
constexpr bool is_null_pointer_type(info type);
constexpr bool is_integral_type(info type);
constexpr bool is_floating_point_type(info type);
constexpr bool is_array_type(info type);
constexpr bool is_pointer_type(info type);
constexpr bool is_lvalue_reference_type(info type);
constexpr bool is_rvalue_reference_type(info type);
constexpr bool is_member_object_pointer_type(info type);
constexpr bool is_member_function_pointer_type(info type);
constexpr bool is_enum_type(info type);
constexpr bool is_union_type(info type);
constexpr bool is_class_type(info type);
constexpr bool is_function_type(info type);
constexpr bool is_reflection_type(info type);

```

```

// associated with [meta.unary.comp], composite type categories
constexpr bool is_reference_type(info type);
constexpr bool is_arithmetic_type(info type);
constexpr bool is_fundamental_type(info type);
constexpr bool is_object_type(info type);
constexpr bool is_scalar_type(info type);
constexpr bool is_compound_type(info type);
constexpr bool is_member_pointer_type(info type);

// associated with [meta.unary.prop], type properties
constexpr bool is_const_type(info type);
constexpr bool is_volatile_type(info type);
constexpr bool is_trivially_copyable_type(info type);
constexpr bool is_trivially_relocatable_type(info type);
constexpr bool is_replaceable_type(info type);
constexpr bool is_standard_layout_type(info type);
constexpr bool is_empty_type(info type);
constexpr bool is_polymorphic_type(info type);
constexpr bool is_abstract_type(info type);
constexpr bool is_final_type(info type);
constexpr bool is_aggregate_type(info type);
constexpr bool is_consteval_only_type(info type);
constexpr bool is_signed_type(info type);
constexpr bool is_unsigned_type(info type);
constexpr bool is_bounded_array_type(info type);
constexpr bool is_unbounded_array_type(info type);
constexpr bool is_scoped_enum_type(info type);

template<reflection_range R = initializer_list<info>>
    constexpr bool is_constructible_type(info type, R&& type_args);
constexpr bool is_default_constructible_type(info type);
constexpr bool is_copy_constructible_type(info type);
constexpr bool is_move_constructible_type(info type);

constexpr bool is_assignable_type(info type_dst, info type_src);
constexpr bool is_copy_assignable_type(info type);
constexpr bool is_move_assignable_type(info type);

constexpr bool is_swappable_with_type(info type_dst, info type_src);
constexpr bool is_swappable_type(info type);

constexpr bool is_destructible_type(info type);

template<reflection_range R = initializer_list<info>>
    constexpr bool is_trivially_constructible_type(info type, R&&
        type_args);
constexpr bool is_trivially_default_constructible_type(info type);
constexpr bool is_trivially_copy_constructible_type(info type);
constexpr bool is_trivially_move_constructible_type(info type);

    constexpr bool is_trivially_assignable_type(info type_dst, info
        type_src);
constexpr bool is_trivially_copy_assignable_type(info type);
constexpr bool is_trivially_move_assignable_type(info type);
constexpr bool is_trivially_destructible_type(info type);

```

```

template<reflection_range R = initializer_list<info>>
    consteval bool is_nothrow_constructible_type(info type, R&&
        type_args);
consteval bool is_nothrow_default_constructible_type(info type);
consteval bool is_nothrow_copy_constructible_type(info type);
consteval bool is_nothrow_move_constructible_type(info type);

    consteval bool is_nothrow_assignable_type(info type_dst, info
        type_src);
consteval bool is_nothrow_copy_assignable_type(info type);
consteval bool is_nothrow_move_assignable_type(info type);

    consteval bool is_nothrow_swappable_with_type(info type_dst, info
        type_src);
consteval bool is_nothrow_swappable_type(info type);

consteval bool is_nothrow_destructible_type(info type);
consteval bool is_nothrow_relocatable_type(info type);

consteval bool is_implicit_lifetime_type(info type);

consteval bool has_virtual_destructor(info type);

consteval bool has_unique_object_representations(info type);

consteval bool reference_constructs_from_temporary(info type_dst, info
    type_src);
consteval bool reference_converts_from_temporary(info type_dst, info
    type_src);

// associated with [meta.unary.prop.query], type property queries
consteval size_t rank(info type);
consteval size_t extent(info type, unsigned i = 0);

// associated with [meta.rel], type relations
consteval bool is_same_type(info type1, info type2);
consteval bool is_base_of_type(info type_base, info type_derived);
    consteval bool is_virtual_base_of_type(info type_base, info
        type_derived);
consteval bool is_convertible_type(info type_src, info type_dst);
    consteval bool is_nothrow_convertible_type(info type_src, info
        type_dst);
consteval bool is_layout_compatible_type(info type1, info type2);
    consteval bool is_pointer_interconvertible_base_of_type(info type-
        _base, info type_derived);

template<reflection_range R = initializer_list<info>>
    consteval bool is_invocable_type(info type, R&& type_args);
template<reflection_range R = initializer_list<info>>
    consteval bool is_invocable_r_type(info type_result, info type, R&&
        type_args);

template<reflection_range R = initializer_list<info>>
    consteval bool is_nothrow_invocable_type(info type, R&& type_args);
template<reflection_range R = initializer_list<info>>

```

```

    consteval bool is_nothrow_invocable_r_type(info type_result, info
        type, R&& type_args);

// associated with [meta.trans.cv], const-volatile modifications
consteval info remove_const(info type);
consteval info remove_volatile(info type);
consteval info remove_cv(info type);
consteval info add_const(info type);
consteval info add_volatile(info type);
consteval info add_cv(info type);

// associated with [meta.trans.ref], reference modifications
consteval info remove_reference(info type);
consteval info add_lvalue_reference(info type);
consteval info add_rvalue_reference(info type);

// associated with [meta.trans.sign], sign modifications
consteval info make_signed(info type);
consteval info make_unsigned(info type);

// associated with [meta.trans.arr], array modifications
consteval info remove_extent(info type);
consteval info remove_all_extents(info type);

// associated with [meta.trans.ptr], pointer modifications
consteval info remove_pointer(info type);
consteval info add_pointer(info type);

// associated with [meta.trans.other], other transformations
consteval info remove_cvref(info type);
consteval info decay(info type);
template<reflection_range R = initializer_list<info>>
    consteval info common_type(R&& type_args);
template<reflection_range R = initializer_list<info>>
    consteval info common_reference(R&& type_args);
consteval info type_underlying_type(info type);
template<reflection_range R = initializer_list<info>>
    consteval info invoke_result(info type, R&& type_args);
consteval info unwrap_reference(info type);
consteval info unwrap_ref_decay(info type);

consteval size_t tuple_size(info type);
consteval info tuple_element(size_t index, info type);
+ consteval bool is_applicable_type(info fn, info tuple);
+ consteval bool is_nothrow_applicable_type(info fn, info tuple);
+ consteval info apply_result(info fn, info tuple);

consteval size_t variant_size(info type);
consteval info variant_alternative(size_t index, info type);

consteval strong_ordering type_order(info type_a, info type_b);

// [meta.reflection.annotation], annotation reflection
consteval vector<info> annotations_of(info item);
consteval vector<info> annotations_of_with_type(info item, info type);
}

```

Add to the front matter in 21.4.1, [meta.syn]:

- 1 Unless otherwise specified, each function, and each specialization of any function template, specified in this header is a designated addressable function ([namespace.std]).
- *

When a function or function template F specified in this header throws a `meta::exception  $E$ ,  $E$ .from() is a reflection representing  $F$ .`
- 2 The behavior of any function specified in namespace `std::meta` is implementation-defined when a reflection of a construct not otherwise specified by this document is provided as an argument.

Add to 21.4.6, [meta.reflection.queries]:

- ```
constexpr bool is_noexcept(info r);
```
- 16 *Returns:* `true` if  $r$  represents a `noexcept` function type or a function with a non-throwing exception specification ([except.spec]). Otherwise, `false`. [ *Note 1:* If  $r$  represents a function template that is declared `noexcept`, `is_noexcept(r)` is still `false` because in general such queries for templates cannot be answered. — *end note* ]
- ```
constexpr bool is_inline(info r);
```
- 17 *Returns:* `true` if r represents an inline namespace ([namespace.def]), variable or variable template declared `inline` (implicitly or explicitly), or function or function template declared `inline` (implicitly or explicitly). Otherwise, `false`.
- ```
constexpr bool is_constexpr(info r);
constexpr bool is_consteval(info r);
```
- 18 *Returns:* `true` if  $r$  represents a variable, variable template, function, or function template declared `constexpr` or `consteval`, respectively. Otherwise, `false`.
- ```
[...]

constexpr vector<info> parameters_of(info r);
```
- 55 ~~Constant When~~ *Throws:* `meta::exception unless` r represents a function or a function type.
- ```
[...]

constexpr info variable_of(info r);
```
- 57 ~~Constant When~~ *Throws:* `meta::exception unless`
- (57.1) —  $r$  represents a parameter of a function  $F$  and

- (57.2) — there is a point  $P$  in the evaluation context for which the innermost non-block scope enclosing  $P$  is the function parameter scope ([`basic.scope.param`]) associated with  $F$ .
- [...]
- ```
consteval info return_type_of(info r);
```
- 59 ~~Constant When~~ *Throws*: `meta::exception` unless either ~~Either~~ r represents a function and `has-type(r)` is `true` or r represents a function type.

Add the new clause [`meta.reflection.scope`] before 21.4.7, [`meta.reflection.access.context`]. The wording for *current-scope* is lifted wholesale from `access_context::current`:

- 1 The function in this subclause can be used to retrieve information about where in the program they are invoked from.
- 2 None of the functions in this clause are addressable functions ([`namespace.std`]).
- 3 Given a program point P , let *eval-point*(P) be the following program point:
 - (3.1) — If a potentially-evaluated subexpression ([`intro.execution`]) of a default member initializer I for a member of a class C ([`class.mem.general`]) appears at P , then a point determined as follows:
 - (3.1.1) — If an aggregate initialization is using I , *eval-point*(Q), where Q is the point at which that aggregate initialization appears.
 - (3.1.2) — Otherwise, if an initialization by an inherited constructor ([`class.inhctor.init`]) is using I , a point whose immediate scope is the class scope corresponding to C .
 - (3.1.3) — Otherwise, a point whose immediate scope is the function parameter scope corresponding to the constructor definition that is using I .
 - (3.2) — Otherwise, if a potentially-evaluated subexpression of a default argument ([`dcl.fct.default`]) appears at P , *eval-point*(Q), where Q is the point at which the invocation of the function ([`expr.call`]) using that default argument appears.
 - (3.3) — Otherwise, if the immediate scope of P is a function parameter scope introduced by a declaration D , and P appears either before the locus of D or within the trailing *requires-clause* of D , a point whose immediate scope is the innermost scope enclosing the locus of D that is not a template parameter scope.
 - (3.4) — Otherwise, if the immediate scope of P is a function parameter scope introduced by a *lambda-expression* L whose *lambda-introducer* appears at point Q , and P appears either within the *trailing-return-type* or the trailing *requires-clause* of L , *eval-point*(Q).
 - (3.5) — Otherwise, if the innermost non-block scope enclosing P is the function parameter scope introduced by a *consteval-block-declaration* ([`dcl.pre`]), a point whose immediate scope is that inhabited by the outermost *consteval-block-declaration* D

containing P such that each scope (if any) that intervenes between P and the function parameter scope introduced by D is either

- (3.5.1) — a block scope or
- (3.5.2) — a function parameter scope or lambda scope introduced by a *consteval-block-declaration*.
- (3.6) — Otherwise, P .

4 Given a scope S , let $\text{ctx-scope}(S)$ be the following scope:

- (4.1) — If S is a class scope or a namespace scope, S .
- (4.2) — Otherwise, if S is a function parameter scope introduced by the declaration of a function, S .
- (4.3) — Otherwise, if S is a lambda scope introduced by a *lambda-expression* L , the function parameter scope corresponding to the call operator of the closure type for L .
- (4.4) — Otherwise, $\text{ctx-scope}(S')$ where S' is the parent scope of S .

5 Let $\text{CURRENT-SCOPE}(P)$ for a point P be a reflection representing the function, class, or namespace whose corresponding function parameter scope, class scope, or namespace scope is $\text{ctx-scope}(S)$, where S is the immediate scope of $\text{eval-point}(P)$.

```
consteval info current_function();
```

6 An invocation of `current_function` that appears at a program point P is value-dependent ([temp.dep.ctxexpr]) if $\text{eval-point}(P)$ is enclosed by a scope corresponding to a templated entity.

7 Let S be $\text{CURRENT-SCOPE}(P)$ where P is the point at which the invocation of *current-scope* lexically appears.

8 *Throws*: `meta::exception` unless S represents a function.

9 *Returns*: S .

```
consteval info current_class();
```

10 An invocation of `current_class` that appears at a program point P is value-dependent ([temp.dep.ctxexpr]) if $\text{eval-point}(P)$ is enclosed by a scope corresponding to a templated entity.

11 Let S be $\text{CURRENT-SCOPE}(P)$ where P is the point at which the invocation of *current-class* lexically appears.

12 *Throws*: `meta::exception` unless S represents either a class or a member function.

13 *Returns*: S if S represents a class. Otherwise, $\text{parent_of}(S)$.

```
consteval info current_namespace();
```

- 14 An invocation of `current_namespace` that appears at a program point P is value-dependent ([`temp.dep.ctxexpr`]) if `eval_point( $P$ )` is enclosed by a scope corresponding to a templated entity.
- 15 Let S be `CURRENT-SCOPE( $P$ )` where P is the point at which the invocation of `current_namespace` lexically appears.
- 16 *Returns:* S is S represents a namespace. Otherwise, a reflection representing the nearest enclosing namespace of the entity represented by S .

Adjust down the now-moved wording from 21.4.7, [`meta.reflection.access.context`]:

- ```
static consteval access_context current() noexcept;
```
- 5 `current` is not an addressable function ([`namespace.std`]).
- 6 ~~Given a program point  $P$ , let `eval_point( $P$ )` be the following program point:~~
- (6.1) ~~— If a potentially-evaluated subexpression ([`intro.execution`]) of a default member initializer  $I$  for a member of a class  $C$  ([`class.mem.general`]) appears at  $P$ , then a point determined as follows:~~
    - (6.1.1) ~~— If an aggregate initialization is using  $I$ , `eval_point( $Q$ )`, where  $Q$  is the point at which that aggregate initialization appears.~~
    - (6.1.2) ~~— Otherwise, if an initialization by an inherited constructor ([`class.inhctor.init`]) is using  $I$ , a point whose immediate scope is the class scope corresponding to  $C$ .~~
    - (6.1.3) ~~— Otherwise, a point whose immediate scope is the function parameter scope corresponding to the constructor definition that is using  $I$ .~~
  - (6.2) ~~— Otherwise, if a potentially-evaluated subexpression of a default argument ([`del.fct.default`]) appears at  $P$ , `eval_point( $Q$ )`, where  $Q$  is the point at which the invocation of the function ([`expr.call`]) using that default argument appears.~~
  - (6.3) ~~— Otherwise, if the immediate scope of  $P$  is a function parameter scope introduced by a declaration  $D$ , and  $P$  appears either before the locus of  $D$  or within the trailing ~~`requires-clause`~~ of  $D$ , a point whose immediate scope is the innermost scope enclosing the locus of  $D$  that is not a template parameter scope.~~
  - (6.4) ~~— Otherwise, if the immediate scope of  $P$  is a function parameter scope introduced by a ~~lambda-expression~~  $L$  whose ~~lambda-introducer~~ appears at point  $Q$ , and  $P$  appears either within the ~~trailing return type~~ or the trailing ~~`requires-clause`~~ of  $L$ , `eval_point( $Q$ )`.~~
  - (6.5) ~~— Otherwise, if the innermost non-block scope enclosing  $P$  is the function parameter scope introduced by a ~~consteval block-declaration~~ ([`del.pre`]), a point whose immediate scope is that inhabited by the outermost ~~consteval block-declaration~~  $D$  containing  $P$  such that each scope (if any) that intervenes between  $P$  and the function parameter scope introduced by  $D$  is either~~

- (6.5.1) — a block scope or
  - (6.5.2) — a function parameter scope or lambda scope introduced by a ~~consteval block declaration~~.
  - (6.6) — Otherwise,  ~~$P$~~ .
- 7 ~~Given a scope  $S$ , let  $ctx\_scope(S)$  be the following scope:~~
- (7.1) — If  ~~$S$~~  is a class scope or a namespace scope,  ~~$S$~~ .
  - (7.2) — Otherwise, if  ~~$S$~~  is a function parameter scope introduced by the declaration of a function,  ~~$S$~~ .
  - (7.3) — Otherwise, if  ~~$S$~~  is a lambda scope introduced by a ~~lambda expression  $L$~~ , the function parameter scope corresponding to the call operator of the closure type for  ~~$L$~~ .
  - (7.4) — Otherwise,  ~~$ctx\_scope(S')$~~  where  ~~$S'$~~  is the parent scope of  ~~$S$~~ .
- 8 An invocation of current that appears at a program point  $P$  is value-dependent ([temp.dep.ctxpr]) if  ~~$eval\_point(P)$~~  is enclosed by a scope corresponding to a templated entity.
- 9 *Returns:* An access\_context whose designating class is the null reflection and whose scope represents the function, class, or namespace whose corresponding function parameter scope, class scope, or namespace scope is  ~~$ctx\_scope(S)$~~ , where  ~~$S$~~  is the immediate scope of  ~~$eval\_point(P)$~~  and is **CURRENT-SCOPE( $P$ )** where  $P$  is the point at which the invocation of current lexically appears.

Fix error-handling in subobjects\_of in [meta.reflection.member.queries]:

- ```
consteval vector<info> subobjects_of(info type, access_context ctx);
```
- 12 ~~Constant When~~ **Throws:** **meta::exception unless** dealias(type) represents a class type that is complete from some point in the evaluation context.

Fix error-handling in reflect_constant_array in 21.4.14, [meta.reflection.array]:

- ```
template <ranges::input_range R>
consteval info reflect_constant_array(R&& r);
```
- 8 Let  $T$  be ranges::range\_value\_t< $R$ >.
- 9 *Mandates:*  $T$  is a structural type ([temp.param]), is\_constructible\_v< $T$ , ranges::range\_reference\_t< $R$ >> is **true**, and is\_copy\_constructible\_v< $T$ > is **true**.
- 10 ~~Constant When~~ **Throws:** **meta::exception unless** reflect\_constant( $e$ ) is a constant subexpression for every element  $e$  of  $r$ .

Change the `data_member_spec` API in 21.4.15, [meta.reflection.define.aggregate]:

```
struct data_member_options {
 struct name-type { // exposition only
 template<class T> requires constructible_from<u8string, T>
 consteval name-type(T &&);

 template<class T> requires constructible_from<string, T>
 consteval name-type(T &&);

 private:
 variant<u8string, string> contents; // exposition only
 };

 + info type;
 optional<name-type> name;
 optional<int> alignment;
 optional<int> bit_width;
 bool no_unique_address = false;
};
```

- 1 The classes `data_member_options` and `data_member_options::name-type` are consteval-only types ([basic.types.general]), and are not structural types ([temp.param]).

```
template<class T> requires constructible_from<u8string, T>
consteval data_member_options::name-type(T&& value);
```

- 2 *Effects:* Initializes `contents` with `u8string(std::forward<T>(value))`.

```
template<class T> requires constructible_from<string, T>
consteval data_member_options::name-type(T&& value);
```

- 3 *Effects:* Initializes `contents` with `string(std::forward<T>(value))`.

[ *Note 2:* The class `name-type` allows the function `data_member_spec` to accept an ordinary string literal (or `string_view`, `string`, etc.) or a UTF-8 string literal (or `u8string_view`, `u8string`, etc.) equally well.

[ *Example 1:*

```
consteval void fn() {
- data_member_options o1 = {.name="ordinary_literal_encoding"};
+ data_member_options o1 = {.type=^^int,
 .name="ordinary_literal_encoding"};
- data_member_options o2 = {.name=u8"utf8_encoding"};
+ data_member_options o2 = {.type=^^char, .name=u8"utf8_encoding"};
}
```

— *end example* ]

— *end note* ]

```
- consteval info data_member_spec(info type,
- data_member_options options);
```

```
+ constexpr info data_member_spec(data_member_options options);
```

#### 4 Constant When:

(4.1) — `dealias(type)` `dealias(options.type)` represents either an object type or a reference type;

(4.2) — if `options.name` contains a value, then:

(4.2.1) — `holds_alternative<u8string>(options.name->contents)` is `true` and `get<u8string>(options.name->contents)` contains a valid identifier ([lex.name]) that is not a keyword ([lex.key]) when interpreted with UTF-8, or

(4.2.2) — `holds_alternative<string>(options.name->contents)` is `true` and `get<string>(options.name->contents)` contains a valid identifier that is not a keyword when interpreted with the ordinary literal encoding;

[ Note 3: The name corresponds to the spelling of an identifier token after phase 6 of translation ([lex.phases]). Lexical constructs like *universal-character-names* [lex.universal.char] are not processed and will cause evaluation to fail. For example, `R"(\u03B1)"` is an invalid identifier and is not interpreted as `"α"`. — end note ]

(4.3) — if `options.name` does not contain a value, then `options.bit_width` contains a value;

(4.4) — if `options.bit_width` contains a value  $V$ , then

(4.4.1) — `is_integral_type(type) || is_enumeration_type(type)` is `true`,

(4.4.2) — `options.alignment` does not contain a value,

(4.4.3) — `options.no_unique_address` is `false`, and

(4.4.4) — if  $V$  equals `0` then `options.name` does not contain a value; and

(4.5) — if `options.alignment` contains a value, it is an alignment value ([basic.align]) not less than `alignment_of(type)`.

#### 5 Returns: A reflection of a data member description ( $T, N, A, W, NUA$ ) ([class.mem.general]) where

(5.1) —  $T$  is the type represented by `dealias(type)` `dealias(options.type)`,

(5.2) —  $N$  is either the identifier encoded by `options.name` or  $\perp$  if `options.name` does not contain a value,

(5.3) —  $A$  is either the alignment value held by `options.alignment` or  $\perp$  if `options.alignment` does not contain a value,

(5.4) —  $W$  is either the value held by `options.bit_width` or  $\perp$  if `options.bit_width` does not contain a value, and

(5.5) —  $NUA$  is the value held by `options.no_unique_address`.

#### 6 [ Note 4: The returned reflection value is primarily useful in conjunction with `define_aggregate`; it can also be queried by certain other functions in `std::meta` (e.g., `type_of`,

```
identifier_of). — end note]
```

Add the various tuple traits in 21.4.16, [meta.reflection.traits]:

```
constexpr info tuple_element(size_t index, info type);
```

9 *Returns:* A reflection representing the type denoted by `tuple_element_t<I, T>` where `T` is the type represented by `dealias(type)` and `I` is a constant equal to `index`.

```
constexpr bool is_applicable_type(info fn, info tuple);
constexpr bool is_nothrow_applicable_type(info fn, info tuple);
```

x *Returns:* `is_applicable_v<F, T>` and `is_nothrow_applicable_v<F, T>`, respectively, where `F` and `T` are the types represented by `dealias(fn)` and `dealias(tuple)`, respectively.

```
constexpr info apply_result(info fn, info tuple);
```

y *Returns:* A reflection representing the type denoted by `apply_result_t<F, T>`, where `F` and `T` are the types represented by `dealias(fn)` and `dealias(tuple)`, respectively.

Change 21.4.17, [meta.reflection.annotation]:

```
constexpr vector<info> annotations_of(info item);
```

1 ~~Constant When~~ *Throws:* `meta::exception unless` `item` represents a type, type alias, variable, function, namespace, enumerator, direct base class relationship, or non-static data member.

[...]

```
constexpr vector<info> annotations_of_with_type(info item, info type);
```

4 ~~Constant When~~ *Throws:* `meta::exception unless`

(4.1) — `annotations_of(item)` is a constant subexpression and

(4.2) — `dealias(type)` represents a type that is complete from some point in the evaluation context.

## § 4 References

- [P1317R2] Aaryaman Sagar and Eric Niebler. 2025-06-19. Remove return type deduction in `std::apply`. <https://wg21.link/p1317r2>
- [P2996R13] Barry Revzin, Wyatt Childers, Peter Dimov, Andrew Sutton, Faisal Vali, Daveed Vandevoorde, and Dan Katz. 2025-06-18. Reflection for C++26.

<https://wg21.link/p2996r13>

[P2996R8] Barry Revzin, Wyatt Childers, Peter Dimov, Andrew Sutton, Faisal Vali, Daveed Vandevorode, Dan Katz. 2024-12-17. Reflection for C++26.

<https://wg21.link/p2996r8>

[P3096R12] Adam Lach, Dan Katz, Walter Genovese. 2025-06-20. Function Parameter Reflection in Reflection for C++26.

<https://wg21.link/p3096r12>

[P3293R3] Peter Dimov, Dan Katz, Barry Revzin, and Daveed Vandevorode. 2025-06-19. Splicing a base class subobject.

<https://wg21.link/p3293r3>

[P3394R4] Wyatt Childers, Dan Katz, Barry Revzin, and Daveed Vandevorode. 2025-06-09. Annotations for Reflection.

<https://wg21.link/p3394r4>

[P3491R3] Wyatt Childers, Peter Dimov, Barry Revzin, and Daveed Vandevorode. 2025-06-20. `define_static_{string,object,array}`.

<https://wg21.link/p3491r3>

[P3560R2] Peter Dimov and Barry Revzin. 2025-06-17. Error Handling in Reflection.

<https://wg21.link/p3560r2>